TAKE-HOME ASSIGNMENT: CINEBOOK - AI-POWERED MOVIE BOOKING PLATFORM

OVERVIEW

Build CineBook, a movie booking platform with an AI chatbot that helps customers find and book movies through natural conversation. The platform serves three types of users:

- Customers browse movies, book tickets, and chat with the AI assistant
- Hall Managers schedule shows for their assigned cinema screens
- Admins oversee everything through a management dashboard

Submission Deadline: 36 hours from receipt
Preferred Tech Stack: Flutter

PART 1: CORE APPLICATION

1.1 User Login & Permissions

Build a secure login system where each user type sees only what they're allowed to access.

- Customer: Browse movies, book tickets, make payments, use chatbot
- Hall Manager: Schedule and manage shows for their assigned screens only
- Admin: Full access to all features and settings

Requirements:
- Secure login with phone number verification (simulated for this assignment)
- Users stay logged in securely across sessions
- The system blocks users from accessing features they shouldn't see

1.2 Customer Experience

Finding Movies

Customers should be able to filter movies by:
- Release date

- Genre (Action, Comedy, Drama, Horror, Sci-Fi, etc.)
- Theatre chain (PVR, INOX, Cinepolis, etc.)
- Screen type (Standard, IMAX, 4DX, Dolby Atmos)
- Format (2D, 3D)
- Language
- Age rating (U, UA, A)

Booking Journey

The booking flow works like this:

Step 1 - Pick a Movie
See what's playing across all theatres

Step 2 - Choose a Show
Filter by date, time, location, and screen type

Step 3 - Select Seats
Visual seat map showing:
- Seat types with different prices:
  - Front Row: Budget-friendly
  - Standard: Regular pricing
  - Premium: Better view, higher price
  - Recliner: Luxury seating, top price
- Which seats are available right now (updates in real-time)
- Color-coded categories for easy identification
- 5-minute hold on selected seats while completing payment

Step 4 - Pay
Simulated payment processing

Step 5 - Get Confirmation
Booking ID and confirmation message

Simulated Payments

Build a test payment system that:
- Accepts test card numbers
- Takes 1-3 seconds to "process" (feels realistic)
- Some test cards always work, some always fail, some randomly fail (to test error handling)
- Creates unique transaction IDs
- Supports refunds

1.3 Hall Manager Experience

Scheduling Shows

Hall managers can add, edit, and remove shows for their assigned screens.

Business Rules the System Must Enforce:

- One theatre can have multiple screens
- Shows can't overlap in the same screen
- At least 30 minutes between shows (for cleaning)
- Can only schedule shows up to 30 days ahead
- Can't change or delete shows that already have bookings

The system should clearly explain what's wrong when a manager tries something that breaks these rules.

1.4 Admin Dashboard

A web-based control center with:

- User Management: View, edit, or disable user accounts; assign roles
- Movie Catalog: Add and update movies (title, description, runtime, cast, posters, trailers)
- Theatre Management: Add theatre chains and locations
- Screen Configuration: Set up screens with seating layouts and equipment types
- Reports: Daily, weekly, and monthly booking and revenue summaries
- Override Powers: Can schedule shows for any screen
- Activity Log: Record of all admin actions for accountability

---

PART 2: AI CHATBOT (MOST IMPORTANT SECTION)

Build an AI assistant that helps customers find and book movies through natural conversation. This is the core of what we're evaluating.

Important: Build the main use cases well. Focus on demonstrating smart architecture rather than handling every edge case.

2.1 What the Chatbot Must Do

A. Have Many Capabilities (20+ Actions)

The chatbot needs at least 20 different actions it can take, organized into logical groups. The AI decides which action to use based on what the customer asks.

Movie-Related Actions:

- Search movies - Find movies matching filters
- Get movie details - Show full info about a specific movie
- Get cast info - Show who's in the movie
- Get reviews - Show what others thought
- Get showtimes - When and where it's playing
- Suggest similar movies - "If you liked X, try Y"
- Show trending - What's popular right now
- Show upcoming - What's coming soon
- List languages - Available language options
- List genres - Available categories

Booking-Related Actions:

- Find theatres - Which theatres are showing a movie
- Get screen info - Details about a specific screen
- Check seat availability - What's open for a show
- Hold seats - Reserve seats temporarily (5 min)
- Release seats - Cancel a hold
- Create booking - Finalize a reservation
- Check booking status - Is my booking confirmed?
- Cancel booking - Cancel an existing booking
- View booking history - What have I booked before?
- Start payment - Begin checkout
- Confirm payment - Complete checkout
- Apply promo code - Add a discount

Plus at least one more group, such as:

- User profile management
- Customer support
- Personalized recommendations

B. Delegate Complex Tasks

When a customer asks something complex like "Book 2 tickets for Inception at PVR Phoenix tomorrow evening," the main chatbot should be able to hand this off to a specialized "booking assistant" that:

- Focuses only on completing the booking
- Searches for the movie, finds available shows, checks seats, and holds the best options
- Reports back with a clear result
- The main chatbot then continues the conversation

Why this matters: This shows the candidate understands how to build AI systems that can break down complex tasks and work efficiently.

C. Handle Long Conversations

The chatbot must stay helpful through extended back-and-forth conversations involving 20+ actions without getting confused.

Example Scenario:

Customer: "I want to watch a sci-fi movie this weekend. Show me what's playing, let me pick one, find theatres near Koramangala with good seats available, and help me book for 2 people. I prefer evening shows with recliner seats. Also check if there are any offers."

The chatbot should smoothly:

1. Search for sci-fi movies
2. Give details when asked about specific movies
3. Show reviews when requested
4. Suggest alternatives if asked
5. Find nearby theatres
6. Filter for recliner seats
7. Show evening showtimes
8. Check seat availability
9. Hold the selected seats
10. Look for promo codes
11. Process payment
12. Confirm the booking
13. Remember preferences for next time

The chatbot must actively manage the conversation context — it shouldn't lose track of what the customer wanted earlier.

D. Chain Actions Together

Actions should flow naturally into each other:

- Searching for a movie gives back a movie ID
- That ID feeds into checking showtimes
- Selecting a showtime gives a show ID
- That ID feeds into checking available seats
- And so on...

Why this matters: Real conversations require connecting multiple steps. The system needs to handle this smoothly.

2.2 Example Conversations

Simple Question:

Customer asks "What movies are releasing this Friday?"

- Chatbot looks up upcoming releases for that date
- Returns a helpful list
- Offers to tell them more about any movie

Complete Booking:

Through natural conversation, the chatbot should:

- Confirm which movie
- Ask about preferred date, time, and location
- Show matching options
- Display available seats with pricing
- Hold selected seats
- Check for discounts
- Handle payment
- Confirm the booking

Each step should feel like talking to a helpful person, not filling out a form.

PART 3: PRODUCTION QUALITY

3.1 Visibility Into What's Happening

Build in the ability to see what's going on:

Activity Logs:
Every important action (especially chatbot actions) should be recorded with: when it happened, what happened, who did it, how long it took, and whether it worked.

Key Metrics:
- How often errors happen and what type
- How long typical conversations are

Request Tracking:
- Follow a single customer interaction from start to finish
- Identify where slowdowns happen

3.2 Handling Failures Gracefully

Automatic Retries:
When something fails temporarily (network glitch, etc.), the system should automatically retry with increasing wait times, not immediately fail.

Circuit Breaker (for payments):
If the payment system starts failing repeatedly, temporarily stop trying and show a helpful message instead of making customers wait through repeated failures.

3.3 Protection Against Abuse

Set limits to protect the system:

- Chat messages: 30 per minute per user
- Booking attempts: 5 per hour per user
- Phone verification requests: 5 per hour per phone number

When limits are hit, tell users when they can try again.

TECHNICAL DETAILS

Technology Choices

Candidates can pick their preferred tools, but should explain why:

- Backend: Node.js, Python, Go, or Java
- Database: PostgreSQL or MongoDB (plus Redis for temporary data like seat holds)
- Frontend: React, Vue, or Angular
- AI: Any major AI provider (OpenAI, Anthropic, etc.)

Critical Requirement: The chatbot logic must be custom-built. Using pre-built AI agent frameworks (LangChain agents, AutoGPT, etc.) is not allowed. We want to see how candidates think about AI architecture.

Required Data Structure

The system needs to track:

- Users — account info, role, contact details
- Movies — title, description, runtime, cast, posters, release date
- Genres — category names
- Theatres — chain, location, address
- Screens — which theatre, screen type, equipment, seating capacity
- Seats — row, number, category, pricing
- Shows — which movie, which screen, when, base price
- Bookings — who booked, which show, payment status, total cost
- Booked Seats — which booking, which seats, price paid
- Payments — amount, status, transaction ID
- Seat Holds — temporary reservations with expiration time
- Admin Activity Log — who did what, when

Documentation

Provide clear documentation of all features and how to use them.

Good luck!